CS 499

Milestone Four: Enhancement Three

Jared Borck

10/4/26

Artifact Description

The artifact I selected for my database enhancement is Travlr Getaways, a full-stack web application that I originally created in CS 465: Full Stack Development during the summer of 2026. The application uses Angular for the frontend, Node.js and Express for the backend and REST API, and MongoDB with Mongoose for data storage. During my previous CS 499 enhancements, I expanded the application with customer ratings and reviews and later added a ranking system that uses weighted scores, merge sort, and a min-heap to rank trips.

For the database enhancement, I focused on improving how the application stores, validates, retrieves, and processes review data. The review schema was strengthened with additional validation and database indexes. I added a compound unique index that prevents the same user from storing more than one review for the same trip. I also added indexes designed around common application queries, including retrieving reviews by trip and finding reviews associated with a user.

Another major improvement was moving review summarization into MongoDB through an aggregation pipeline. Previously, the application retrieved individual reviews and performed more of the processing in Node.js. The enhanced version uses MongoDB operations such as $group, $avg, $sum, $project, and $round to calculate review counts, average rating values, and weighted trip scores before returning the summarized data to the application.

Justification for Inclusion

I selected Travlr Getaways for the database category because the project already relied on MongoDB, but there were several opportunities to improve how the database was being used. Earlier versions of the application mainly treated the database as a place to store and retrieve records. This enhancement allowed me to demonstrate additional database skills involving data integrity, validation, indexing, aggregation, and query efficiency.

One of the most important improvements was moving the one-review-per-user-per-trip rule into the database itself. Before this enhancement, the application controller checked whether a user had already reviewed a trip before creating another review. This worked during normal use, but application-level validation alone does not completely guarantee that duplicate records cannot be created. For example, two requests could theoretically reach the server close enough together that both pass the application check before either record is saved. The new compound unique index on the trip code and user ID makes MongoDB the final authority and prevents the duplicate record from being stored.

I also updated the backend controller so MongoDB duplicate-key errors are handled properly. A duplicate-key error now produces a meaningful conflict response instead of being treated as an unknown server failure. Mongoose validation errors are also returned as client validation errors.

This allows the application layer and database layer to work together while still providing clear responses to the user.

The review model was also improved with stronger validation. Rating values must be whole numbers from one through five, review text has a maximum length, and important relationship fields such as the trip code and user ID cannot be changed after a review is created. These changes improve the consistency and integrity of the stored data.

Database indexes were another important part of the enhancement. I created compound indexes for queries that the application performs frequently. One index supports retrieving reviews for a particular trip in newest-first order, while another supports retrieving recent reviews associated with a particular user. The unique user-and-trip index also supports duplicate prevention. I verified the query behavior using MongoDB's explain() functionality and confirmed that MongoDB could use the index rather than relying on an unrestricted collection scan.

The largest change involved the rankings feature. Instead of retrieving every review and performing all of the summarization in Node.js, MongoDB now performs an aggregation pipeline. Reviews are grouped by trip, and MongoDB calculates the number of reviews and average values for the five rating categories. It also calculates the weighted score used by the existing ranking system. This reduces the amount of raw review data that needs to be transferred from the database into the application when only summary information is required.

This improvement also allowed me to preserve the Algorithms and Data Structures enhancement from the previous milestone. MongoDB is responsible for database operations such as grouping and calculating averages, while the existing merge sort and min-heap remain responsible for ordering the summarized trip results and selecting the Top 3 trips. This creates a clearer separation between database processing and application-level algorithms.

Course Outcome Coverage

This enhancement most directly supports Course Outcome 4, which focuses on using appropriate techniques, skills, and computing tools to implement solutions that provide value and accomplish specific goals. MongoDB and Mongoose are no longer being used only for basic storage. The enhanced application uses schema validation, compound indexes, uniqueness constraints, aggregation operations, and query analysis to improve the way the application manages and processes its data.

The work also supports Course Outcome 5 because several of the changes improve the integrity and security of application data. The unique compound index prevents duplicate review records even if application-level validation fails or simultaneous requests occur. Important ownership and relationship fields are protected from being modified after creation, and invalid rating values are rejected before they can be stored.

There is also some additional support for Course Outcome 3 because database design involves trade-offs similar to other computing problems. Adding indexes can improve read performance and enforce important constraints, but indexes also require additional storage and maintenance when records are added or changed. Moving aggregation into MongoDB also changes where processing occurs. In this case, it made sense because MongoDB can summarize the review collection before returning results to Node.js instead of sending every individual review across the database boundary.

My overall course-outcome plan has not changed significantly. The first enhancement focused most strongly on software engineering and security, the second focused on algorithms and data structures, and this enhancement completes the database category. Together, the three enhancements provide evidence across several of the program outcomes while also showing how different computer science concepts can work together within the same application.

Reflection on the Enhancement Process

One of the main things I learned during this enhancement was the difference between application-level validation and database-level data integrity. Before this milestone, I already had logic that prevented a user from intentionally submitting a second review for the same trip. However, that rule only existed in the application code. Adding a unique compound index showed me why important data rules should also be enforced as close to the data as possible. The application can provide a convenient first check, while the database provides the final guarantee.

I also gained a better understanding of indexes. Adding an index is relatively simple, but deciding which fields should be indexed requires thinking about how the application actually accesses its data. I used the queries already being performed by the application to decide which compound indexes were useful. I then used MongoDB's query explanation tools to examine how a review query was executed rather than simply assuming that the index would be used. This made the database optimization more measurable and helped me understand the connection between database design and application performance.

The aggregation pipeline was the most significant technical part of the enhancement. Before this change, the application could retrieve individual review records and calculate values after the data reached Node.js. With the new approach, MongoDB groups the reviews and calculates counts and averages before the application receives the results. I learned how database operations such as $group, $avg, $sum, $project, and $round can work together to turn a collection of individual records into summarized information.

One consideration was making sure the database enhancement did not replace the work completed during the Algorithms and Data Structures milestone. The ranking feature already used custom merge sort and min-heap implementations, and removing those algorithms would have weakened the previous enhancement. Instead, I separated the responsibilities. MongoDB

now prepares and summarizes the review information, and the existing algorithms continue to rank the results. This allowed the project to demonstrate both database processing and application-level algorithm design without one enhancement replacing the other.

Testing also showed that the enhancement preserved the application's existing behavior. After moving the review calculations into MongoDB, the rankings still appeared in the same order and produced the same weighted results as before. The API also returned the new category averages and database-processing information successfully. Using MongoDB's explain() functionality provided additional confirmation that the database indexes were being used for the intended query pattern.

Overall, this enhancement helped me move beyond using MongoDB only for basic CRUD operations. I gained experience with database constraints, schema validation, indexing, aggregation, query analysis, and the trade-offs involved in deciding where data processing should occur. The finished version of Travlr Getaways now uses the database more effectively while preserving the software engineering and algorithm improvements completed during the earlier milestones.